- **Dataset:** all image/mask examples available to the experiment.
- **Training split:** examples used to calculate loss and update weights.
- **Validation split:** examples used to compare model performance during training.
- **Batch size:** number of examples processed together before an optimizer update (the file uses variants such as 16, and later GPU/CPU variants differ).
- **DataLoader:** loads examples in batches and can shuffle training data.

Why not use validation data to train?

If the model repeatedly updates weights directly on the validation examples, the validation score stops being an independent check. A final, untouched test set would be even better for reporting performance.

4. The main model: U-Net with a ResNet34 encoder

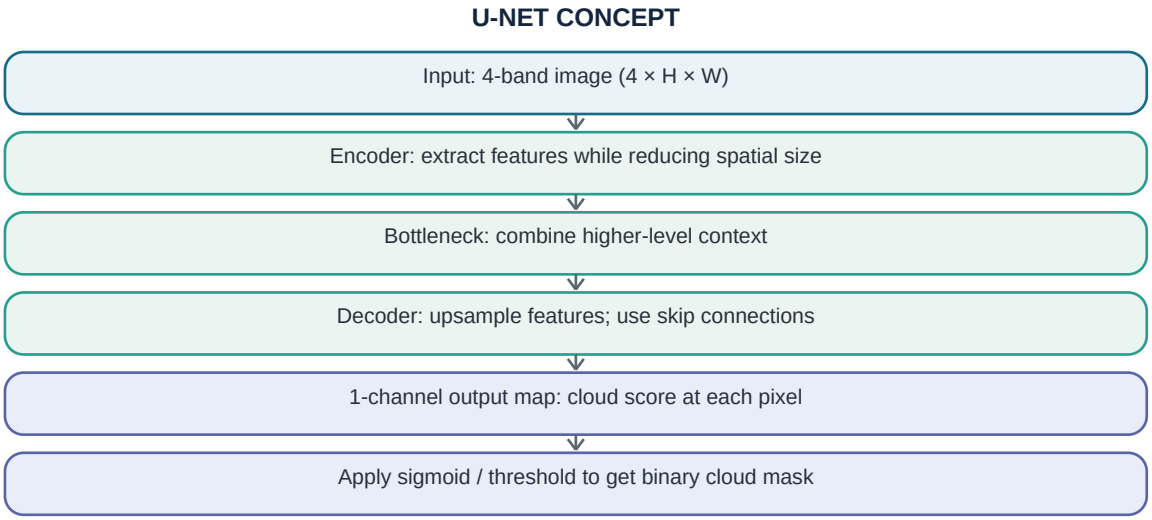
The main cloud-segmentation network is configured with a U-Net architecture and a ResNet34 encoder pretrained on ImageNet. The model accepts four input channels and produces one output channel for cloud-vs-background segmentation.

4.1 What is semantic segmentation?

Image classification gives one label for an entire image—for example, “cloudy”. Semantic segmentation gives a label to every pixel. For this project, the goal is to create a mask where cloud pixels are 1 and non-cloud pixels are 0.

4.2 U-Net in simple language

U-Net has a contracting path (encoder) and an expanding path (decoder). The encoder gradually compresses the image into feature maps that represent patterns such as edges, textures and larger structures. The decoder upsamples these features back toward the original image resolution. Skip connections carry high-resolution features from encoder stages to decoder stages, helping recover object boundaries.



4.3 Why ResNet34?

ResNet34 is the encoder backbone. “ResNet” means Residual Network. Residual/skip connections inside ResNet help train deeper networks by allowing information and gradients to travel through identity-style paths. The code requests ImageNet-pretrained weights, so the encoder starts from features learned on a large general image dataset rather than random initialization.

The model is adapted for four channels. Standard RGB pretrained models expect three input channels, so the segmentation library must construct/adapt the first layer to accept red, green, blue and NIR. Pretraining can help, but the fourth channel is not an ImageNet RGB channel; the adaptation method matters.

```
Unet(
    encoder_name='resnet34',
    encoder_weights='imagenet',
    in_channels=4,
    classes=1,
    activation=None
)
```

Setting	Meaning
encoder_name='resnet34'	Use ResNet34 to extract features.
encoder_weights='imagenet'	Initialize encoder from pretrained weights when available.
in_channels=4	Four input bands rather than RGB only.
classes=1	One foreground score channel: cloud.

Setting	Meaning
activation=None	Return logits; apply sigmoid when calculating probabilities or use a logits-aware loss.

4.4 Why use one output channel?

This is a binary segmentation problem: cloud versus not-cloud. One logit per pixel is enough. During inference, a sigmoid converts a logit z into a score between 0 and 1: sigmoid(z) = 1 / (1 + e^{-z}). The code then applies a threshold, typically 0.5, to decide whether each pixel is foreground.

5. Loss functions and evaluation metrics

A loss function measures how different the prediction is from the ground-truth mask. The optimizer tries to reduce this loss. A metric is a score used to describe the quality of predictions; metrics are especially useful because a loss value is not always easy to interpret.

5.1 Binary Cross-Entropy (BCE) with logits

BCE is designed for binary labels. It penalizes a prediction that assigns low probability to the correct class. The “with logits” version combines the sigmoid operation and cross-entropy calculation in a numerically stable way. Because the model returns raw logits, this loss is a natural fit.

Intuition: if the true pixel is cloud (1), the model should assign a high cloud score; if the true pixel is background (0), it should assign a low cloud score.

5.2 Dice score and Dice loss

The Dice score measures overlap between the predicted cloud mask and the true mask. It is especially useful in segmentation when the foreground class occupies a smaller part of an image.

$$\text{Dice} = 2 \times |\text{Prediction} \cap \text{Ground Truth}| \,/\, (|\text{Prediction}| + |\text{Ground Truth}|)$$

Dice loss is often calculated as 1 – Dice. A score near 1 means strong overlap; a score near 0 means little overlap. Implementations add a small smoothing constant to avoid division by zero.

5.3 Combined loss

```
total_loss = 0.5 * BCEWithLogitsLoss + 0.5 * DiceLoss
```

The file combines BCE and Dice loss with equal weights in the described configuration. BCE encourages correct per-pixel probabilities; Dice loss encourages overall overlap. Combining them can balance local pixel correctness and mask-level overlap.

5.4 Intersection over Union (IoU)

IoU compares the overlap of predicted and true foreground areas to their union:

$$\text{IoU} = |\text{Prediction} \cap \text{Ground Truth}| \,/\, |\text{Prediction} \cup \text{Ground Truth}|$$

IoU ranges from 0 to 1. A higher value indicates greater overlap. Dice and IoU are related but not identical. The code computes metrics by thresholding the predicted probabilities, commonly at 0.5, then comparing binary masks.

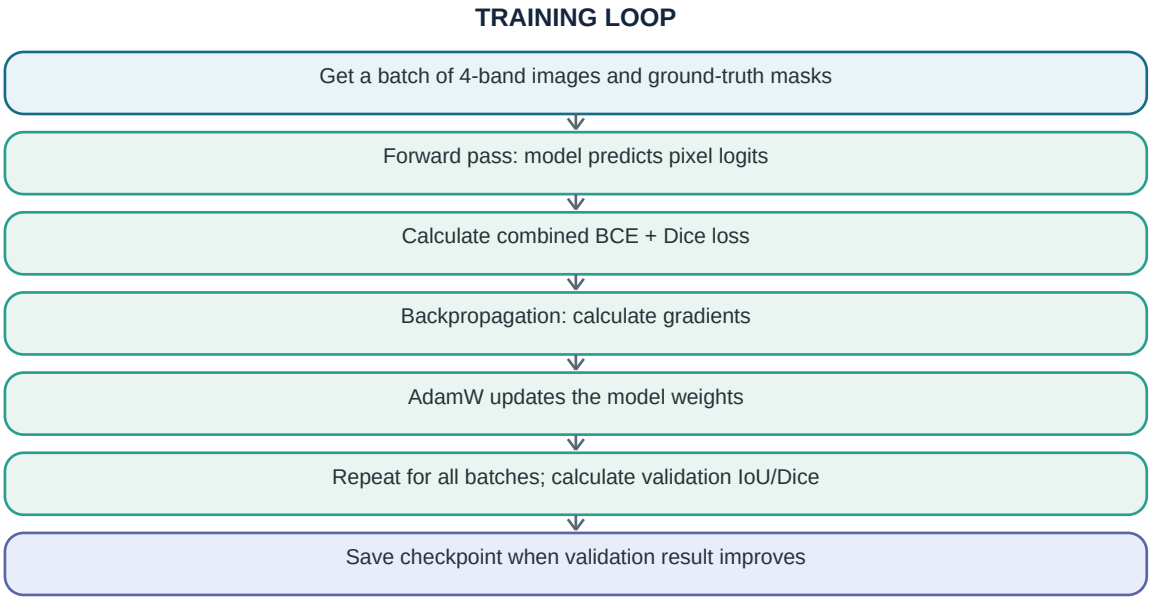
5.5 Pixel accuracy

Pixel accuracy is the fraction of all pixels classified correctly. It can be misleading if background pixels greatly outnumber cloud pixels: a model might classify most pixels as background and still get high accuracy. Therefore Dice and IoU are important companion metrics.

Measure	What it answers	Higher is better?
BCE loss	How good are the pixel probabilities compared with labels?	No — lower is better
Dice loss	How far is mask overlap from ideal?	No — lower is better
Dice score	How much do predicted and true masks overlap?	Yes
IoU	How much is the intersection relative to the union?	Yes
Pixel accuracy	What fraction of all pixels are correct?	Usually, but interpret with class balance

6. How the model was trained

Training is an iterative process. The model makes a prediction, the loss compares it with the ground-truth mask, automatic differentiation calculates gradients, and the optimizer updates model weights. This repeats across batches and epochs.



6.1 Important training terms

Term	Beginner explanation
Epoch	One complete pass through the training split.
Batch	A group of examples processed together.
Forward pass	The model calculates predictions from the input.
Loss	A number measuring prediction error.
Backpropagation	Calculates how each trainable parameter contributed to error, via gradients.
Optimizer step	Changes trainable parameters using the gradients.
Validation	Measures performance on held-out examples without updating weights.
Checkpoint	A saved copy of model parameters, such as best_cloud_unet.pth.

6.2 Optimizer: AdamW

The code uses AdamW with a learning rate around 5 × 10■■ and weight decay around 10■■ in the main configuration. AdamW adapts the update scale for individual parameters using running estimates of gradients and squared gradients. Weight decay discourages excessively large weights and can help regularization.

Learning rate is the size of the parameter update. If too high, training may be unstable; if too low, learning can be slow. Weight decay is a regularization setting, not a direct measure of model accuracy.

6.3 Learning-rate scheduler

A CosineAnnealingLR scheduler gradually changes the learning rate following a cosine-shaped schedule. The main configuration uses T_max=10 and eta_min=10■■, alongside a 10-epoch training setup. The exact effect depends on when the scheduler is stepped in the loop.

6.4 Training settings seen in the file

Setting	Value / interpretation
Train/validation split	Approximately 85% / 15%
Batch size	A main variant uses 16; other variants differ by hardware.
Epochs	A main setup uses 10 epochs.
Optimizer	AdamW
Learning rate	About 0.0005 in the main configuration
Weight decay	About 0.0001
Scheduler	CosineAnnealingLR; T_max=10; eta_min=0.000001
Checkpoint	Best validation IoU saved as best_cloud_unet.pth
Other implementation details	Some variants use mixed precision, pin_memory and multiple data-loading workers.

6.5 What is mixed precision?

Mixed precision uses lower-precision floating-point arithmetic for selected operations to reduce GPU memory use and sometimes improve speed. Training code may use autocast and a gradient scaler. It is a performance technique, not a different segmentation algorithm.

6.6 What does “best checkpoint” mean?

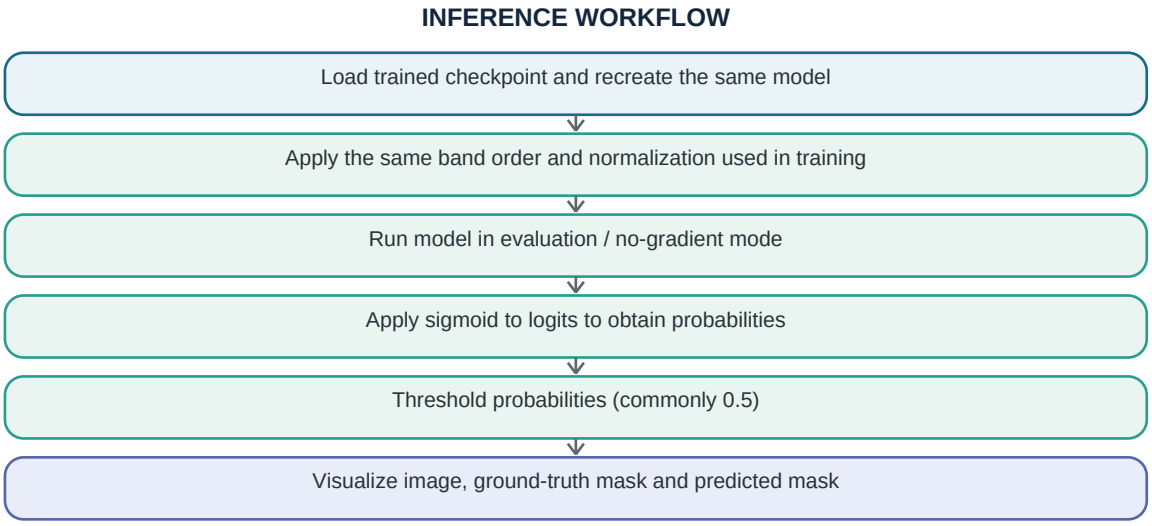
During validation, the code computes a score such as IoU and saves the model when that score improves. This is preferable to blindly saving only the last epoch, because validation performance can fluctuate. The filename best_cloud_unet.pth typically stores the learned weights; the model architecture and preprocessing must also be recreated correctly to use it.

Practical code warning

The file contains a call equivalent to `os.kill(os.getpid(), 9)`. That forcefully terminates the running Python process. If the script is run from top to bottom, execution may stop at that line; run sections intentionally or remove/guard this command during normal development.

7. From trained model to cloud mask

After training, the model is used for inference: it receives an image and predicts a score for every pixel. These scores are converted into a binary mask so downstream code can work with cloud and non-cloud regions.



Why evaluation mode matters

Calling model.eval() changes the behavior of layers such as dropout and batch normalization for inference. Disabling gradient tracking with torch.no_grad() reduces memory use because no backward pass is required.

Threshold trade-off

A threshold of 0.5 is a conventional choice, not a universal optimum. A lower threshold can mark more pixels as cloud (possibly increasing recall but adding false positives); a higher threshold can mark fewer pixels (possibly reducing false positives but missing clouds). Choose the threshold using validation data and the intended application.

Visual comparison should show at least: the original image or a band composite, the ground-truth mask, the predicted mask, and optionally an overlay. Visual inspection is useful, but quantitative metrics on held-out data are still needed.

8. Route planning: binary dilation, risk grids and A*

The route planner is a separate algorithmic component. It treats an image or risk map like a grid. A cell represents a location. The route starts at a start cell and attempts to reach a destination while minimizing a cost.

8.1 Binary dilation

Binary dilation expands foreground regions in a binary mask. If a cloud mask is dilated, nearby pixels around the cloud region are also marked. The intent is to add a margin rather than plan a path directly along a detected boundary. The margin depends on the structuring element and number of dilation iterations.

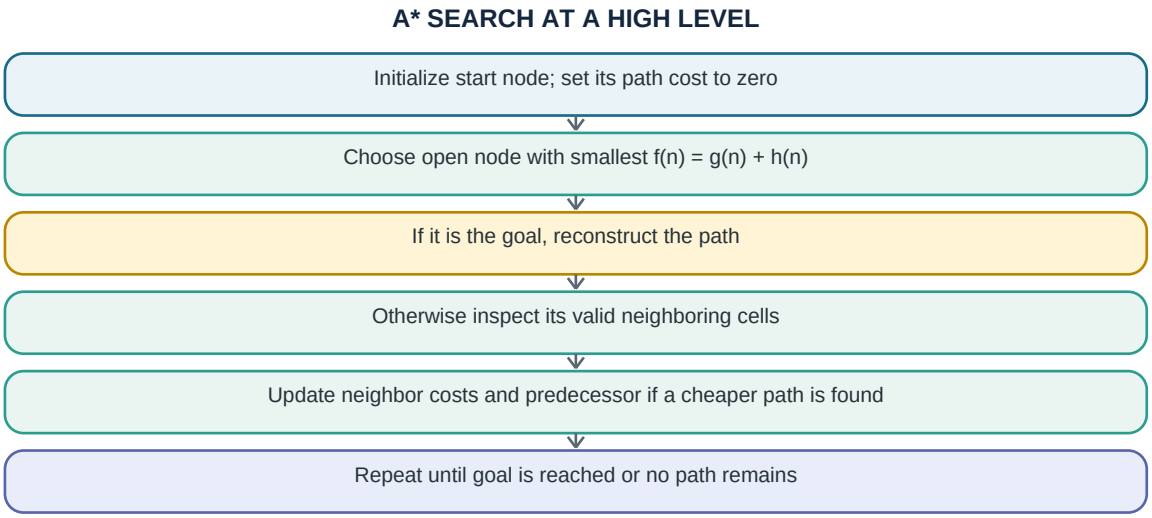
Dilation is a mathematical morphology operation. It does not prove that the expanded area is a safe or correct buffer for aircraft; a real buffer would need calibrated geospatial scale and aviation rules.

8.2 A* search algorithm

A* is a graph-search algorithm that finds a low-cost path from a start node to a goal node. On a grid, each cell is a node and adjacent cells are neighbors. The algorithm uses:

- **g(n):** the known cost from the start to the current node.
- **h(n):** a heuristic estimate of the remaining cost to the goal.
- **f(n) = g(n) + h(n):** estimated total cost through the node.

The implementation uses 8-direction movement: horizontal/vertical moves cost about 1, while diagonal moves cost about 1.414 (√2). The heuristic is Euclidean distance, which estimates straight-line distance to the goal.



8.3 Risk-weighted A*

A risk-aware variant adds a penalty for traversing a cell with a high risk score. Conceptually, the step cost may be:

```
step cost = movement cost + risk weight × cell risk
```

A larger risk weight makes the search more strongly avoid risky cells. The code also blocks cells above a risk threshold around 0.95 in one variant. This is only meaningful if the risk grid is meaningful and scaled correctly.

8.4 Adaptive routing

The code tries several risk penalties, such as 100, 50, 20, 5 and 0, until it finds a route that fits a maximum path-length or “max_fuel” constraint. This is an experimental heuristic: path length in grid steps is not the same as aircraft fuel consumption, and the chosen penalty is not a certified trade-off.

Potential implementation issue to review

The code reportedly contains a neighbor-cost comparison using a default such as `gscore.get(neighbor, 0)`. For a previously unseen node, a default of zero is usually incorrect because every non-negative candidate cost would fail

to improve it. Standard A* implementations generally use infinity (or explicitly treat an unseen neighbor as improvable). Verify and fix this before relying on the route output.

9. Cyclone-related experiments and their limits

The file contains multiple cyclone-related experiments, but they should be described carefully. A cloud segmentation model does not automatically detect cyclones. Cyclone detection requires cyclone labels or a validated detection/classification target, suitable imagery and an evaluation protocol.

9.1 Synthetic storm rectangle and Gaussian risk map

Some sections insert a rectangular region into a cloud mask to imitate a storm region. Other sections create a risk field from a synthetic rectangle and smooth it with a Gaussian filter. Gaussian filtering spreads/smooths a signal so the risk region changes gradually instead of having a perfectly sharp edge.

These are simulated examples for demonstrating route avoidance. They are not detections made by a trained cyclone detector. In a presentation, call them “synthetic storm/risk scenarios” unless the code is replaced with a real, labeled cyclone detection method.

9.2 Infrared imagery and altitude proxy

The script downloads INSAT-3D infrared cyclone images and includes a calculation resembling `cloud_altitude_ft = ir_normalized * 55000`. This maps normalized pixel intensity directly to a number between roughly 0 and 55,000 feet. That is not a physically validated altitude retrieval. Infrared brightness temperature, radiometric calibration, cloud-top retrieval and atmospheric assumptions are needed for actual cloud-top height estimates.

Therefore describe this as an illustrative altitude proxy, not an actual altitude-aware aviation route planner. Real route planning would require calibrated geospatial coordinates, altitude layers, wind and weather forecasts, aircraft performance and aviation constraints.

9.3 ResNet18 cyclone intensity regression

The code defines a CycloneIntensityDataset and a ResNet18-based regression model. ResNet18 is a convolutional neural network originally designed for image classification. The final layer is replaced with a regression head that outputs one continuous number.

Component	What it does
Grayscale infrared input	Represents image intensity in one channel.
ResNet18 backbone	Extracts image features using convolutional layers and residual connections.
Regression head	Fully connected layers reduce extracted features to one numeric output.
MSE loss	Penalizes squared difference between predicted and target numeric values.
AdamW	Updates model parameters to reduce training loss.

9.4 The key pseudo-label issue

The target values are generated from an image statistic such as the average intensity/coldness of a central image region, using a formula like `knots = 35 + (core_coldness / 255) * 100`. This is a pseudo-label: a value created by a heuristic, not an observed best-track wind speed. The model therefore learns to reproduce the formula's output from images. It has not been shown to predict real cyclone wind speed.

A defensible real-world experiment would pair each image with verified cyclone intensity labels (for example, documented maximum sustained winds), ensure images and labels are time-aligned, separate storms between train and test sets, compare against a baseline, and report MAE/RMSE on held-out real labels.

10. Airport suitability neural network

The file includes an AirportDecisionNet feed-forward neural network. It takes five numeric features and returns a score between 0 and 1 through a sigmoid output.

Feature	Meaning intended by the code
Aircraft–airport distance	Approximate distance from the aircraft position to candidate airport.
Airport–storm distance	Approximate distance from airport to the storm region.
Elevation	Airport elevation feature.
Facility type score	Numeric encoding of airport/facility type.
Fuel remaining	Estimate or ratio of fuel remaining.

10.1 Network architecture

```
5 input features
  → Linear(5, 128) + BatchNorm + ReLU + Dropout
  → Linear(128, 64) + BatchNorm + ReLU
  → Linear(64, 32) + ReLU
  → Linear(32, 1) + Sigmoid
  → score in [0, 1]
```

ReLU is a non-linear activation function: ReLU(x) = max(0, x). Batch normalization helps stabilize intermediate activations. Dropout randomly disables some units during training to reduce overfitting. The final sigmoid maps the output to a number between 0 and 1.

10.2 What is the target label?

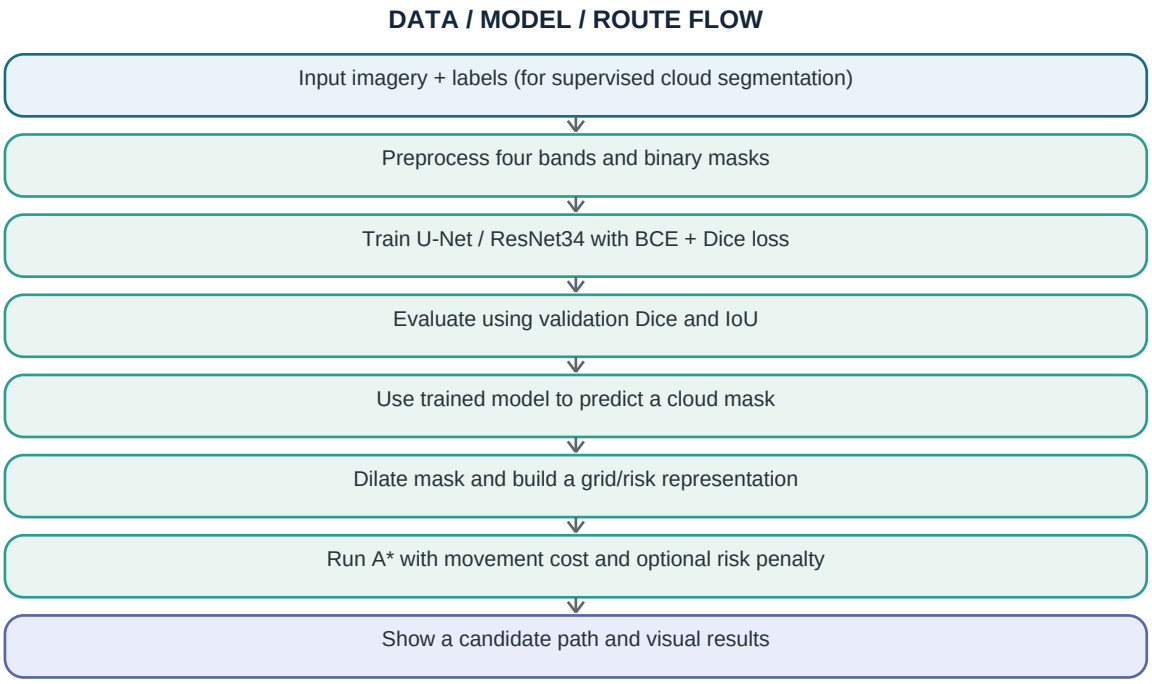
The code constructs synthetic targets from a hand-written rule, approximately combining fuel ratio, weather/storm-distance ratio and facility ratio with weights such as 0.45, 0.35 and 0.20, then applying penalties. The network is trained to imitate these generated scores.

This is supervised learning on synthetic labels, but the labels are not real examples of successful/failed diversions. The network may simply learn to approximate the formula that generated its training targets. It should not be described as a validated airport-selection system.

10.3 Distance approximation

The code approximates distances from latitude/longitude differences using a factor around 60. This is a rough approximation, not a robust great-circle distance calculation. For realistic geographic work, use a proper geodesic calculation, correct coordinate units, route geometry and authoritative airport data.

11. End-to-end workflow and algorithm summary



Algorithm summary

Algorithm / method	Category	Purpose in this project	Key point
U-Net	Deep learning architecture	Pixel-wise cloud segmentation	Encoder–decoder with skip connections.
ResNet34	CNN backbone	Feature extraction inside U-Net	Pretrained encoder; adapted for 4 channels.
BCEWithLogitsLoss	Loss function	Pixel-wise binary classification error	Works directly with raw logits.
Dice loss	Overlap-based loss	Encourage cloud-mask overlap	Useful with imbalanced foreground/background.
AdamW	Optimizer	Update model parameters	Adaptive updates plus weight decay.
CosineAnnealingLR	Learning-rate schedule	Change learning rate during training	Cosine-shaped schedule.
ResNet18 regression	Deep learning model	Estimate intensity-like number	Current labels are heuristic pseudo-labels.
MSE	Regression loss	Penalize numeric prediction errors	Large errors are penalized quadratically.
Binary dilation	Morphology	Expand mask around cloud areas	Adds a grid-space margin.
Gaussian filter	Image processing	Smooth synthetic risk field	Does not validate risk estimates.
A*	Graph search	Find a low-cost grid path	Uses g(n), h(n), f(n)=g+h.
MLP / AirportDecisionNet	Neural network	Score candidate airport features	Training labels are rule-generated.

12. What is strong, what is experimental, and what should improve?

This section is important for a technically honest presentation. It shows that you understand the difference between a working code prototype and a validated model.

Area	Current limitation indicated by code	Improvement
Cloud segmentation	Need measured results on a separate test set; preprocessing and channel adaptation must be consistent.	Report test IoU/Dice, use reproducible splits, compare against baseline, inspect failure cases.
Cyclone detection	The U-Net is a cloud segmenter, not a cyclone detector by itself.	Train/evaluate a dedicated cyclone classifier or detector with verified labels.
Cyclone intensity	Targets are generated from image intensity, not verified wind speeds.	Use real intensity labels, time alignment, storm-level splits, MAE/RMSE.
Synthetic storm map	A rectangle and smoothed risk field are simulations.	Use documented cyclone tracks, calibrated geospatial weather/risk layers.
Altitude estimate	Normalized IR brightness is scaled to 55,000 ft without a physical retrieval model.	Use a validated cloud-top-height retrieval product and uncertainty estimates.
A* implementation	Review unseen-node g-score default; 0 is usually wrong.	Use infinity for unseen nodes; test with known maps and compare with Dijkstra.
Flight safety	Grid path length and a risk score are not aircraft dynamics or legal routing.	Do not use operationally; integrate validated forecasts, aircraft constraints, NOTAMs, ATC and certified procedures.
Airport score	Training targets are generated from a rule formula.	Use real diversion/airport data and domain-expert validation; evaluate calibrated outcomes.

Additional engineering checks

- Make train/validation/test splits reproducible with a fixed random seed.
- Check for data leakage: images from the same scene or storm should not appear across train and test splits if that would inflate performance.
- Record library versions, dataset source/version, preprocessing choices and model configuration.
- Save model weights and any normalization or threshold settings needed for inference.
- Inspect class balance and report precision/recall as well as Dice/IoU when useful.
- Never place API keys or Kaggle credentials in a public script. If a real credential was included in the shared file, revoke/rotate it and remove it from the file and its history.

Bottom line

The best-supported central claim is: “We trained a U-Net with a ResNet34 encoder for binary cloud segmentation and explored how a risk grid could be used with A* to produce a candidate path. The file also contains experimental cyclone-intensity and airport-scoring models, but their current labels and risk values are partly synthetic, so they need real-world validation before any safety claim.”

13. Presentation script for beginners

You can adapt this script for a 2–3 minute project explanation. Avoid claiming that the prototype predicts real cyclone wind speed or produces safe flight routes.

Opening / problem: “Our project explores how satellite imagery and algorithms can support cloud analysis and experimental flight-route planning. A large image contains many pixels, so we use semantic segmentation to identify cloud regions automatically. We then explore how a route-search algorithm can avoid cells assigned higher risk.”

Data: “For cloud segmentation, we load red, green, blue and near-infrared bands from the 38Cloud dataset. Each image is paired with a ground-truth binary mask. We normalize the image values and split the data into training and validation sets.”

Model: “The main model is U-Net with a ResNet34 encoder. The encoder extracts image features, while the decoder reconstructs a pixel-level prediction. Skip connections help preserve spatial details. The model takes four channels and produces one cloud score per pixel.”

Training: “We train by comparing predictions with the mask using a combined Binary Cross-Entropy with logits and Dice loss. AdamW updates the weights. A cosine learning-rate scheduler changes the learning rate over training. We monitor validation IoU and save the best checkpoint.”

Evaluation: “We use Dice score and Intersection over Union to measure overlap between predicted cloud regions and the ground-truth mask. These metrics are more informative for segmentation than pixel accuracy alone, especially when background pixels dominate.”

Route planning: “For route planning, we represent the area as a grid. A* searches from a start cell to a destination. It combines the cost already travelled, g, with a heuristic estimate to the goal, h. A risk penalty can be added to the cost so the search prefers lower-risk cells.”

Other experiments and limitations: “The file also contains a ResNet18 regression experiment and an airport-scoring network. In the current version, some labels and storm-risk regions are synthetic, so these are demonstrations rather than validated cyclone or aviation safety models. Our next step is to evaluate with verified real-world labels and calibrated geospatial weather data.”

One-sentence conclusion

“The core implementation combines deep-learning-based cloud segmentation with algorithmic grid pathfinding, while cyclone-intensity and airport-selection components remain experimental and require real-data validation.”

14. Likely presentation questions and beginner answers

Why did you use U-Net?

Because the goal is pixel-level segmentation. U-Net combines contextual features from an encoder with a decoder that reconstructs a detailed mask, using skip connections to retain spatial information.

Why ResNet34?

It is a CNN backbone that extracts features. Pretrained weights provide a useful starting point and residual connections help optimization.

Why four input channels?

The cloud dataset supplies red, green, blue and near-infrared bands. NIR adds information beyond ordinary RGB.

Why not use ordinary accuracy only?

If most pixels are background, a model can achieve high pixel accuracy while missing clouds. Dice and IoU measure overlap of the cloud mask.

What is BCEWithLogitsLoss?

A stable binary classification loss that works directly with raw model logits, without needing a separate sigmoid before the loss.

What is Dice loss?

An overlap-oriented loss that encourages predicted cloud regions to match the ground-truth mask.

What does AdamW do?

It updates model parameters based on gradients, using adaptive learning-rate behavior and decoupled weight decay.

What is an epoch?

One pass through all examples in the training split.

What is A*?

A graph-search algorithm that selects nodes using f(n)=g(n)+h(n), where g is cost so far and h estimates the remaining cost.

Is A* a machine-learning model?

No. A* is a search algorithm. It uses a cost map and a heuristic rather than learning weights from training examples.

Does the U-Net detect cyclones?

Not by itself. The U-Net is trained for cloud-vs-background segmentation. A separate cyclone model and verified cyclone labels are needed for a cyclone detection claim.

Does the ResNet18 model predict real wind speed?

Not as currently described. Its targets are generated from image brightness heuristics, so it learns pseudo-labels, not verified wind-speed values.

Is the route safe for real aircraft?

No. It is a research prototype. Safe routing needs validated meteorological data, aircraft constraints, geospatial calculations, air-traffic rules and certification.

What is the main future improvement?

Evaluate cloud segmentation on an untouched test set and replace synthetic cyclone/risk/airport labels with verified, time-aligned real-world data.

15. Final checklist before presenting

- Be able to explain the difference between classification, segmentation and regression.
- Explain the four input bands and why the mask is binary.
- Explain U-Net's encoder, decoder and skip connections in one or two sentences.
- Know why the training loss combines BCE and Dice, and what IoU measures.
- Know the difference between training and validation data.
- Explain A* using f(n) = g(n) + h(n), and clarify that A* is not a trained model.
- Call synthetic storm/risk areas and pseudo-labels what they are—simulated or heuristic.
- Do not claim the current altitude proxy is a real cloud-top-height measurement.
- Do not claim the airport score predicts real diversion success.
- Remove or guard the process-killing command before running the complete script.
- Rotate any exposed API/Kaggle credential if the file contained a genuine secret.

Glossary

Term	Plain-language meaning
CNN	Convolutional neural network: learns spatial patterns in images.
Segmentation	Predicting a class for each pixel.
Encoder	Part of a network that extracts increasingly abstract features.
Decoder	Part of a segmentation network that reconstructs a spatial output.
Logit	Raw model score before sigmoid or softmax.
Ground truth	The reference label used to train or evaluate a model.
Overfitting	When a model fits training examples but performs poorly on new data.
Pseudo-label	A target created by a heuristic or another model instead of verified ground truth.
Heuristic	A practical estimate used to guide a search or decision.
Risk grid	A grid assigning a numeric cost or risk value to each cell.
Geospatial calibration	Making sure image pixels and measurements correspond correctly to real-world locations/scales.

End of guide